PRODUCT / DATA / MACHINE LEARNING / DELIVERY

Benchline

Recommendation evaluation and release workbench

75-page build contract

From deterministic events to a measured, reversible model release

ENGINEERING PLAN / EDITION 1

Pablo Williams / 04 September 2026

DECISION RECORD + DELIVERY STANDARD

How to read this plan

The plan is arranged as a build contract. Product intent comes first, system and experience design follow, and the final third defines proof. Page numbers are stable so pull requests and interview notes can cite a decision precisely.

DECISIONS

- Decision statements use imperative language and are binding for the first public release.
- Delivery items describe repository artifacts, not future intentions.
- Evidence items define the test or observable result required for acceptance.

DELIVERY CONTRACT

1. Maintain a traceability matrix linking plan pages to code, tests, screenshots, benchmark files, and architecture decisions.

2. Record deliberate deviations in an ADR rather than silently changing scope.

ACCEPTANCE EVIDENCE

1. Every P0 capability has at least one automated check and one visible product path.

2. The final audit reports implemented, partially implemented, and deferred items without marketing language.

Executive summary

Most portfolio recommenders stop at model training. Benchline focuses on the expensive part: operating a multi-stage ranking product safely. It combines realistic offline evaluation with a control plane for exploration, comparison, release, and diagnosis.

DECISIONS

- Optimize for demonstrable engineering judgment over infrastructure theatre.
- Use a three-stage pipeline: candidate retrieval, learned ranking, and constraint-aware re-ranking.
- Expose model quality, system performance, and product guardrails in the same release decision.

DELIVERY CONTRACT

1. Six coherent surfaces: Command Center, Explorer, Experiments, Registry, Feature Health, and Delivery.

2. A typed recommendation API and a Python evaluation package sharing versioned contracts.

ACCEPTANCE EVIDENCE

1. The challenger must beat a popularity baseline on NDCG@10 with a bootstrap interval reported.

2. The serving path must publish measured p50 and p95 latency from a repeatable local load test.

The product opportunity

Recommendation teams often split context across notebooks, registry screens, orchestration logs, and BI tools. Benchline compresses the release conversation into a single evidence chain while preserving links to underlying artifacts.

DECISIONS

- Design for a small platform team supporting multiple product surfaces.
- Make offline-to-online mismatch and exposure bias first-class risks.
- Prefer decisions and exceptions over charts that merely report activity.

DELIVERY CONTRACT

1. A release gate that joins quality, safety, freshness, and latency checks.

2. An investigation path from a metric regression to cohort, feature, model, and request evidence.

ACCEPTANCE EVIDENCE

1. A reviewer can answer ‘why is this model safe to promote?’ without reading source code.

2. Failure states show an owner, consequence, and next action.

Primary users

The application serves four users with different questions: the ML engineer improving relevance, the data engineer protecting features, the product analyst assessing impact, and the release owner controlling production risk.

DECISIONS

- Default the interface to the release owner’s current decision, then allow technical drill-down.
- Use shared vocabulary across pages: surface, cohort, model, run, gate, and evidence.
- Avoid persona theatre; each user is represented by a concrete workflow.

DELIVERY CONTRACT

1. Role-oriented shortcuts and context-preserving navigation.

2. Read access for all seeded roles; promotion and rollback controls visibly permissioned.

ACCEPTANCE EVIDENCE

1. Each primary workflow completes in under five deliberate interactions after landing.

2. Terminology is defined in product copy and the glossary rather than assumed.

Jobs to be done

Benchline is useful when a team needs to understand recommendation behaviour, compare a challenger, approve a release, or investigate a quality regression. These jobs define the navigation and the API boundaries.

DECISIONS

- Make ‘explain this slate’ the core Explorer job.
- Make ‘compare against the champion’ the core Experiments job.
- Make ‘promote with evidence’ the core Registry job.

DELIVERY CONTRACT

1. Saved cohorts, reproducible request snapshots, experiment comparisons, and signed release records.

2. Deep links preserve user, surface, model, and run context.

ACCEPTANCE EVIDENCE

1. No workflow depends on hidden hover states.

2. A fresh reviewer can identify the next action from each page header.

Product principles

The product should feel like a tool built by people who operate ranking systems. Calm density, explicit provenance, conservative defaults, and sharp exception handling matter more than decorative animation.

DECISIONS

- Evidence before confidence: always show the source, window, and model version behind a metric.
- Progressive disclosure: summaries lead to inspectable rows, then raw evidence.
- Reversible operations: promotions use staged gates and rollback remains adjacent.

DELIVERY CONTRACT

1. A visual system based on ink, warm neutral surfaces, cobalt actions, and signal colours reserved for state.

2. Plain-language empty, loading, stale, partial, and failure states.

ACCEPTANCE EVIDENCE

1. Colour is never the sole carrier of status.

2. Motion is restrained and disabled under reduced-motion preferences.

Success framework

Success spans ranking quality, catalogue health, serving performance, and operating confidence. A single relevance number cannot justify a production release.

DECISIONS

- Primary quality metric: NDCG@10 on a temporal holdout.
- Guardrails: coverage, long-tail exposure, creator concentration, freshness, error rate, and p95 latency.
- Operational metric: time required to explain and safely reverse a release.

DELIVERY CONTRACT

1. Metric definitions include numerator, denominator, exclusions, window, owner, and failure action.

2. A release scorecard records values, thresholds, comparison direction, and provenance.

ACCEPTANCE EVIDENCE

1. All headline metrics are computed by tested functions.

2. No percentage is shown without its comparison population or time window.

Scope and non-goals

Version one proves the control plane and evaluation discipline on a laptop. It does not pretend to operate internet-scale infrastructure or train a billion-parameter model.

DECISIONS

- Include deterministic synthetic marketplace data, baselines, hybrid retrieval, an interpretable scorer, policy re-ranking, and a measured API.
- Model distributed components through explicit interfaces and deployment manifests.
- Exclude payment flows, real identity data, GPU training, and claims of live customer traffic.

DELIVERY CONTRACT

1. A production-shaped monorepo with replaceable local adapters.

2. Clear seams for Kafka, warehouse, feature store, vector index, registry, and observability backends.

ACCEPTANCE EVIDENCE

1. The README states what is real, simulated, and designed for replacement.

2. Deferred capabilities appear in the roadmap, never as enabled UI controls.

Assumptions and constraints

The build must be impressive on a recruiter’s laptop: no cloud account, proprietary data, or API key should be required for the primary demo. Determinism and bounded dependencies are therefore architectural requirements.

DECISIONS

- Seed all randomness and version generated data.
- Keep the default dataset small enough for CI while preserving user, item, time, and exposure structure.
- Make optional infrastructure additive rather than mandatory.

DELIVERY CONTRACT

1. Documented hardware envelope, setup time, and supported runtime versions.

2. Graceful fallback when Redis, Kafka, or external telemetry are absent.

ACCEPTANCE EVIDENCE

1. Cold clone to working demo in under ten minutes on a supported machine.

2. Repeated benchmark runs remain within a documented tolerance.

System context

Benchline sits between behavioural data producers, offline data systems, model development, and product serving. The web application is an operating surface over these contracts, not the system of record for raw events.

DECISIONS

- Separate control-plane state from data-plane inference.
- Treat model artifacts and feature definitions as versioned inputs.
- Represent external systems with ports so local adapters and production adapters share contracts.

DELIVERY CONTRACT

1. A C4 context diagram and a repository-level dependency map.

2. Interfaces for event source, offline store, online store, model registry, and telemetry sink.

ACCEPTANCE EVIDENCE

1. Dependency direction is enforced by tests or lint rules.

2. The serving package does not import UI or orchestration modules.

Reference architecture

The reference path ingests events, materializes features, builds retrieval indexes, trains and evaluates models, registers candidates, and serves ranked slates through a version-aware API.

DECISIONS

- Keep the request path synchronous and bounded; move training, indexing, and reporting off path.
- Load immutable model bundles by alias and retain the previous champion for rollback.
- Attach request, feature-set, candidate-set, and model identifiers to every trace.

DELIVERY CONTRACT

1. Architecture diagram covering batch, stream, control, and serving paths.

2. Local compose topology and production-oriented Kubernetes manifests.

ACCEPTANCE EVIDENCE

1. A contract test follows one event into features and one request through a ranked response.

2. A rollback test proves the previous bundle can be restored without rebuilding.

Bounded contexts

The monorepo is divided by responsibility: contracts, data generation, features, retrieval, ranking, evaluation, serving, control plane, and web experience. Ownership should be visible from paths and APIs.

DECISIONS

- Use shared schema packages only for stable cross-boundary types.
- Keep numerical evaluation code independent of the web framework.
- Do not build a generic platform abstraction before two concrete implementations require it.

DELIVERY CONTRACT

1. Top-level directories with READMEs explaining purpose and allowed dependencies.

2. Architecture tests for forbidden imports and schema drift.

ACCEPTANCE EVIDENCE

1. A new engineer can locate the owner of an endpoint or metric without repository archaeology.

2. Circular package dependencies fail CI.

Behavioural event ingestion

Events are the raw material of both features and evaluation. A recommendation impression must be distinguishable from a click, completion, save, and purchase, with enough context to reconstruct exposure.

DECISIONS

- Adopt an append-only event envelope with event_id, occurred_at, received_at, actor, session, surface, and schema_version.
- Make impressions first-class and attach slate position and model metadata.
- Reject impossible timestamps and quarantine unknown schema versions.

DELIVERY CONTRACT

1. JSON Schema contracts, fixtures, validation code, and a dead-letter example.

2. A deterministic event generator with realistic sessions and delayed feedback.

ACCEPTANCE EVIDENCE

1. Duplicate event IDs are idempotent.

2. Contract tests cover valid, late, duplicate, malformed, and forward-version events.

Identity and session stitching

Ranking features need stable entities without leaking personal identity. The demo uses synthetic opaque identifiers and a documented merge policy for anonymous and signed-in sessions.

DECISIONS

- Never store names, emails, or device fingerprints.
- Use an append-only identity link with effective timestamps.
- Prevent future identity merges from rewriting historical training examples.

DELIVERY CONTRACT

1. Synthetic user, anonymous actor, session, and consent-state tables.

2. A point-in-time stitching function with test vectors.

ACCEPTANCE EVIDENCE

1. Historical examples remain unchanged after a later merge.

2. Opt-out events remove the actor from future feature materialization.

Catalogue and taxonomy

The fictional catalogue contains learning products with topic, level, duration, format, price, quality, freshness, and creator attributes. These fields support meaningful relevance and policy trade-offs.

DECISIONS

- Use a controlled hierarchical taxonomy with stable IDs.
- Separate editorial quality from behavioural popularity.
- Version availability and price so historical evaluation sees the correct state.

DELIVERY CONTRACT

1. Catalogue schema, seed generator, taxonomy file, and slowly changing history.

2. Validation for orphaned categories, invalid prices, unavailable items, and duplicate slugs.

ACCEPTANCE EVIDENCE

1. Every recommended item is eligible at request time.

2. The catalogue generator produces a documented distribution, not uniform noise.

Offline feature store

Offline features support reproducible training and analysis. Every feature value carries entity keys, event timestamp, creation timestamp, and feature-view version.

DECISIONS

- Store columnar snapshots locally and define a warehouse adapter contract.
- Partition by event date without encoding business logic in paths.
- Treat feature definitions as code reviewed alongside tests.

DELIVERY CONTRACT

1. User affinity, item quality, popularity, novelty, price sensitivity, and cross features.

2. A feature catalogue describing owners, freshness, null policy, and leakage risk.

ACCEPTANCE EVIDENCE

1. Rebuilding a snapshot from the same inputs yields identical hashes.

2. Null, range, uniqueness, and freshness expectations run before training.

Online feature access

Serving requires a small, bounded set of fresh features. The local implementation uses an in-process cache adapter while keeping key and freshness semantics compatible with Redis.

DECISIONS

- Fetch features in bulk by entity and version.
- Return freshness metadata and distinguish missing from zero.
- Enforce a strict deadline with a documented fallback feature vector.

DELIVERY CONTRACT

1. OnlineStore protocol, memory adapter, optional Redis adapter, and metrics.

2. Warm-up and cache invalidation hooks tied to feature-set releases.

ACCEPTANCE EVIDENCE

1. Missing features trigger fallback without a 500 response.

2. Feature retrieval time appears separately in request traces.

Point-in-time training joins

Training rows may use only information available before the prediction timestamp. Point-in-time correctness is essential because popularity, price, availability, and user affinity all change.

DECISIONS

- Use as-of joins by entity and event timestamp.
- Apply a feature availability delay where ingestion latency matters.
- Fail closed when a feature lacks temporal semantics.

DELIVERY CONTRACT

1. A reusable point-in-time join module and leakage test dataset.

2. Documentation showing a naive join failure beside the correct result.

ACCEPTANCE EVIDENCE

1. A deliberately planted future value never appears in historical features.

2. Training manifests record source and feature snapshot hashes.

Popularity baseline

A credible evaluation begins with simple baselines. Popularity by surface and recent window provides an interpretable floor and a robust fallback when personalization signals are unavailable.

DECISIONS

- Compute decayed engagement from impressions, clicks, saves, and completions.
- Exclude future and ineligible interactions.
- Maintain global and taxonomy-conditioned variants.

DELIVERY CONTRACT

1. A deterministic baseline model with serialized parameters.

2. Baseline metrics by overall population, cold users, and sparse categories.

ACCEPTANCE EVIDENCE

1. The baseline is reproducible and served through the same model interface.

2. Every learned model comparison names its exact baseline version.

Collaborative retrieval

Implicit-feedback matrix factorization provides a strong, inspectable retrieval baseline without requiring GPU infrastructure. Confidence weights reflect action strength and recency.

DECISIONS

- Train on exposure-aware implicit feedback.
- Tune factors and regularization using a temporal validation window.
- Filter previously completed and currently unavailable items after retrieval.

DELIVERY CONTRACT

1. A lightweight matrix-factorization implementation or pinned library wrapper.

2. Saved user and item factors plus training metadata.

ACCEPTANCE EVIDENCE

1. Recall@100 and catalogue coverage beat the popularity candidate set for warm users.

2. Unknown users fall back predictably.

Two-tower retrieval design

The advanced retrieval path maps users and items into a shared embedding space. It demonstrates architecture and evaluation discipline without overstating production scale.

DECISIONS

- Use categorical embeddings and compact numerical towers.
- Train with sampled negatives drawn from exposed but unengaged items plus hard category negatives.
- Normalize vectors and score by dot product.

DELIVERY CONTRACT

1. A PyTorch design specification, training CLI contract, and optional implementation milestone.

2. Exported item embeddings with schema and model fingerprints.

ACCEPTANCE EVIDENCE

1. Retrieval metrics are compared at equal candidate budgets.

2. The public demo remains functional if the optional neural model is not trained.

Candidate index

The candidate index turns item vectors into bounded retrieval. The default adapter uses exact search for deterministic CI; an approximate-nearest-neighbour port documents production substitution.

DECISIONS

- Version indexes independently but require model compatibility metadata.
- Build indexes offline and swap atomically.
- Over-fetch before eligibility and diversity filters.

DELIVERY CONTRACT

1. Exact cosine index, manifest format, health check, and ANN adapter interface.

2. Index build and query benchmarks.

ACCEPTANCE EVIDENCE

1. A mismatched model/index fingerprint fails startup.

2. Recall against exact search is measurable for future ANN adapters.

Candidate blending

No single retriever handles warm users, exploration, trends, and cold start. Candidate blending combines collaborative, content, popular, and fresh pools before ranking.

DECISIONS

- Record source and source rank for every candidate.
- Allocate per-source budgets by request context.
- Deduplicate by item ID while retaining all provenance.

DELIVERY CONTRACT

1. Configurable blend policy and diagnostic contribution report.

2. A candidate-set snapshot available from Explorer.

ACCEPTANCE EVIDENCE

1. The final set reaches its requested size when eligible catalogue supply allows.

2. Source starvation and dominance are surfaced as diagnostics.

Learning-to-rank model

The ranker estimates engagement utility from user, item, context, and retrieval features. Gradient-boosted trees provide strong tabular performance, explainability, and fast CPU inference.

DECISIONS

- Train a pairwise or list-aware objective where library support is stable.
- Group examples by request or session to preserve ranking structure.
- Calibrate scores only when an interface requires probability semantics.

DELIVERY CONTRACT

1. Feature matrix builder, model wrapper, feature manifest, and training report.

2. Global and per-request explanation contributions.

ACCEPTANCE EVIDENCE

1. Model serialization is deterministic within pinned versions.

2. No feature used in training is absent or semantically different at serving.

Policy-aware re-ranking

The final slate balances predicted utility with diversity, freshness, creator concentration, and business eligibility. Policy is explicit so product choices are not hidden inside model weights.

DECISIONS

- Apply hard eligibility filters before soft objectives.
- Use a greedy maximal-marginal-relevance style objective with configurable weights.
- Record every position change and the policy responsible.

DELIVERY CONTRACT

1. Re-ranker module, policy configuration schema, and counterfactual slate view.

2. Constraint tests for category repetition, creator caps, and freshness floors.

ACCEPTANCE EVIDENCE

1. Every output item has a traceable pre-rank and post-rank position.

2. Impossible constraints return a degraded-but-valid slate with warnings.

Cold-start strategy

Cold start is not one case. New users, new items, anonymous sessions, sparse categories, and empty contexts need distinct behaviours that remain useful and honest.

DECISIONS

- Classify cold-start state explicitly in the request context.
- Blend editorial quality, contextual popularity, content similarity, and measured exploration.
- Protect new-item exposure with quality and eligibility thresholds.

DELIVERY CONTRACT

1. Cold-start decision table and seeded scenarios.

2. Dedicated evaluation slices and Explorer labels.

ACCEPTANCE EVIDENCE

1. A brand-new user receives a diverse eligible slate.

2. Cold-item exposure can be audited without lowering global safety constraints.

Feedback loops and bias

Logged interactions reflect what prior models exposed, not unrestricted preference. Benchline must state this limitation and reduce obvious bias in training and evaluation.

DECISIONS

- Log propensities for simulated policies.
- Weight or stratify analyses by exposure where justified.
- Separate observed engagement from causal impact claims.

DELIVERY CONTRACT

1. Exposure diagnostics, position-bias chart, and methodology note.

2. A simulation comparing naive and propensity-aware estimates.

ACCEPTANCE EVIDENCE

1. The UI never labels offline uplift as causal revenue impact.

2. Known bias sources appear beside experiment conclusions.

Canonical data model

Stable entities anchor the product: users, sessions, items, creators, events, impressions, feature values, datasets, experiments, models, aliases, deployments, and release decisions.

DECISIONS

- Use opaque string IDs with readable prefixes.
- Carry created_at and updated_at where mutation exists; use effective intervals for temporal state.
- Represent provenance through foreign keys and immutable hashes.

DELIVERY CONTRACT

1. Entity relationship diagram and SQL migrations.

2. Seed fixtures that exercise every relationship.

ACCEPTANCE EVIDENCE

1. Foreign-key and uniqueness constraints are tested.

2. Deleting control-plane records cannot orphan evidence.

Event contract

The event schema is a public boundary and receives stricter governance than internal tables. Producers can evolve additive fields while consumers reject ambiguous breaking changes.

DECISIONS

- Version the envelope and payload independently only when needed.
- Use UTC ISO timestamps and validated enums.
- Include request_id and recommendation metadata on exposed actions.

DELIVERY CONTRACT

1. Machine-readable schemas, generated TypeScript/Python types, and compatibility tests.

2. Examples for impression, click, save, completion, and purchase.

ACCEPTANCE EVIDENCE

1. Examples validate in both languages.

2. A breaking fixture demonstrably fails compatibility CI.

Feature definitions

A feature is more than a column name. Each definition includes entity, dtype, computation, source, timestamp semantics, default, freshness target, owner, and leakage classification.

DECISIONS

- Give feature sets semantic versions.
- Deprecate through dual-read windows rather than silent replacement.
- Allow request-time context features only through a separate namespace.

DELIVERY CONTRACT

1. Feature registry file and rendered catalogue in the UI.

2. Validation and drift rules generated from definitions where practical.

ACCEPTANCE EVIDENCE

1. Training and serving fingerprints match before promotion.

2. Unused, undocumented, or expired features fail a governance check.

API conventions

The HTTP API should read like a dependable internal platform: explicit versions, typed errors, idempotent mutations, request correlation, bounded pagination, and stable timestamps.

DECISIONS

- Prefix public contracts with /api/v1.
- Use problem-details style errors with code, message, request_id, and safe details.
- Require idempotency keys for promotion and rollback operations.

DELIVERY CONTRACT

1. OpenAPI document, generated examples, and a small typed client.

2. Consistent pagination and filtering vocabulary.

ACCEPTANCE EVIDENCE

1. Contract tests validate representative responses against schemas.

2. Errors never expose stack traces or filesystem paths.

Recommendation endpoint

POST /api/v1/recommendations returns a ranked slate plus bounded evidence suitable for product rendering and operational debugging.

DECISIONS

- Accept user, session, surface, limit, context, and optional model alias.
- Return item, rank, score, reason codes, policy adjustments, model version, feature version, and request ID.
- Keep deep feature contributions behind a debug permission.

DELIVERY CONTRACT

1. Serving handler with validation, deadlines, fallback, and trace spans.

2. Golden requests covering warm, anonymous, cold, sparse, and degraded paths.

ACCEPTANCE EVIDENCE

1. The same request and bundle produce the same ordered result.

2. Latency and fallback state are observable without leaking sensitive features.

Experiment endpoints

Experiment APIs expose immutable run metadata and computed evidence. The browser never calculates authoritative metrics from raw arrays.

DECISIONS

- Separate experiment definition, execution, and result resources.
- Treat completed run metrics as immutable.
- Support comparisons only when evaluation protocol and dataset compatibility checks pass.

DELIVERY CONTRACT

1. List, detail, compare, and evidence-export endpoints.

2. Typed filters for status, model family, dataset, owner, and date.

ACCEPTANCE EVIDENCE

1. Incompatible experiments return a clear reason instead of a misleading delta.

2. Evidence export hashes match displayed run identifiers.

Registry and release endpoints

Model aliases are operational pointers. Promotion changes the champion only after gates pass and a release record captures who, what, why, and how to reverse it.

DECISIONS

- Represent champion and challenger as aliases, not copied artifacts.
- Use compare-and-swap semantics to prevent concurrent promotion races.
- Require a human release note even in the demo.

DELIVERY CONTRACT

1. Registry list/detail, gate evaluation, promote, rollback, and audit endpoints.

2. Immutable release decision and alias history tables.

ACCEPTANCE EVIDENCE

1. A stale expected champion version causes a conflict.

2. Rollback restores the previous version and appends an audit event.

Authentication and authorization

The public demo uses a clearly labelled local identity, while the architecture supports external OIDC. Authorization is enforced in the service, not only by hidden buttons.

DECISIONS

- Define viewer, analyst, release-manager, and administrator roles.
- Keep debug features and mutations permissioned.
- Use secure-by-default cookie and token guidance for deployed adapters.

DELIVERY CONTRACT

1. Policy module, route guards, UI capability map, and authorization tests.

2. A demo identity switcher that never implies real enterprise SSO.

ACCEPTANCE EVIDENCE

1. Direct API calls cannot bypass role restrictions.

2. Audit events record actor and decision for privileged operations.

Information architecture

The interface is organised around operating decisions rather than technical layers. The sidebar names user jobs; secondary navigation appears only within a selected object.

DECISIONS

- Command Center summarises current health and decisions.
- Explorer explains individual slates.
- Experiments, Registry, Feature Health, and Delivery own their respective operating workflows.

DELIVERY CONTRACT

1. Desktop sidebar, mobile drawer, command menu, breadcrumbs, and stable deep links.

2. A route map documented beside the component hierarchy.

ACCEPTANCE EVIDENCE

1. All primary pages are reachable by keyboard.

2. Refreshing a detail route preserves its state.

Command Center

The landing page answers three questions: is serving healthy, is a release waiting, and where does attention belong. It avoids a collage of generic KPI cards.

DECISIONS

- Lead with one operational sentence and the current champion.
- Group evidence into Quality, Delivery, and Data lanes.
- Use an attention queue ordered by consequence and recency.

DELIVERY CONTRACT

1. A measured request-volume/latency figure, quality trend, release-gate summary, and recent decisions.

2. Contextual links into the exact failing cohort, feature, or deployment.

ACCEPTANCE EVIDENCE

1. Every warning names an owner and action.

2. Healthy states remain compact so anomalies dominate attention.

Recommendation Explorer

Explorer is the signature demo. It lets a reviewer choose a user archetype and surface, generate a slate, and inspect the chain from candidate sources through ranking and policy changes.

DECISIONS

- Show results as a human-readable product slate, not a database table.
- Pair each result with concise reason codes and provenance.
- Provide before/after ordering and feature contributions on demand.

DELIVERY CONTRACT

1. Scenario presets, search, request controls, ranked cards, evidence drawer, and raw JSON copy.

2. A compare mode for champion versus challenger.

ACCEPTANCE EVIDENCE

1. All displayed reasons derive from response fields.

2. Empty and degraded scenarios remain explainable.

Experiments workspace

Experiments turns model evaluation into a reviewable decision. It prioritises comparison validity, uncertainty, and slices over leaderboard theatre.

DECISIONS

- Require a common protocol before showing direct deltas.
- Display confidence intervals beside point estimates.
- Keep guardrail regressions visible even when the primary metric improves.

DELIVERY CONTRACT

1. Run list, comparison canvas, metric definitions, slice table, and evidence export.

2. An ablation view connecting feature groups to quality and latency.

ACCEPTANCE EVIDENCE

1. A reviewer can identify the winning model and its caveats in under one minute.

2. Statistically uncertain changes are labelled inconclusive.

Model Registry

Registry presents models as deployable bundles with lineage, not filenames. The release path is deliberate, reviewable, and reversible.

DECISIONS

- Show alias, model family, dataset, feature set, code revision, metrics, and signature together.
- Keep promotion in a side panel with gates and release note.
- Place rollback history beside the active champion.

DELIVERY CONTRACT

1. Model inventory, bundle detail, lineage graph, gate evaluation, promotion dialog, and audit timeline.

2. Permission and concurrency failure states.

ACCEPTANCE EVIDENCE

1. Promotion is impossible while a blocking gate fails.

2. The visible champion always matches the serving alias returned by the API.

Feature Health

Feature Health joins contract status, freshness, distribution shift, nulls, and training-serving consistency. It should help an engineer diagnose a model problem before retraining blindly.

DECISIONS

- Organise by feature view and entity.
- Show freshness and quality expectations before distribution charts.
- Link incidents to affected models and surfaces.

DELIVERY CONTRACT

1. Health summary, feature table, drift detail, lineage, and ownership information.

2. A point-in-time correctness check and materialisation run history.

ACCEPTANCE EVIDENCE

1. Stale and missing are visually and semantically distinct.

2. A feature incident identifies blast radius and fallback behaviour.

Delivery

Delivery shows whether the recommendation path is meeting its operational contract. It combines deployment history, SLOs, traces, and bounded logs without impersonating a full observability vendor.

DECISIONS

- Anchor the page on service level objectives.
- Correlate deployments with latency and error changes.
- Sample request traces with redacted payloads.

DELIVERY CONTRACT

1. SLO scorecard, latency distribution, deployment timeline, trace waterfall, and structured log viewer.

2. A rollback drill with recorded recovery time.

ACCEPTANCE EVIDENCE

1. Figures distinguish measured local evidence from illustrative production targets.

2. Sensitive user and feature data never appears in logs.

Visual language

Benchline should feel editorial and operational: warm off-white canvas, charcoal navigation, cobalt actions, compact typography, and diagrams that resemble engineering documents rather than neon AI concept art.

DECISIONS

- Use a single sans family with tabular numerals and a restrained display weight.
- Reserve gradients for data encoding only, not page decoration.
- Use borders, spacing, and typography before shadows.

DELIVERY CONTRACT

1. Design tokens for colour, type, spacing, radius, elevation, motion, and chart palettes.

2. A small icon set with consistent stroke weight.

ACCEPTANCE EVIDENCE

1. No glassmorphism, floating blobs, excessive pills, or rainbow status treatments.

2. Screens remain legible when printed or viewed in grayscale.

Accessibility

Accessibility is a release requirement. Dense technical software must remain navigable without a mouse, understandable without colour, and usable at enlarged text sizes.

DECISIONS

- Target WCAG 2.2 AA for authored UI.
- Use native controls and semantic landmarks before ARIA.
- Provide focus return for dialogs and drawers.

DELIVERY CONTRACT

1. Skip link, logical headings, visible focus, labelled charts, live regions, and reduced motion.

2. Automated axe checks plus keyboard walkthroughs.

ACCEPTANCE EVIDENCE

1. Zero serious automated accessibility violations on primary views.

2. Command menu, filters, dialogs, tabs, and tables complete a keyboard test script.

Responsive behaviour

The mobile layout is not a compressed desktop dashboard. It prioritises status, current decision, and essential controls while moving detail into drawers and stacked disclosures.

DECISIONS

- Define intentional layouts at 390, 768, 1024, and 1440 pixels.
- Convert wide tables to labelled record cards or horizontal regions with explicit affordance.
- Keep primary actions reachable without covering content.

DELIVERY CONTRACT

1. Responsive navigation, chart resizing, card ordering, drawers, and touch targets.

2. Visual regression references for one mobile and one desktop viewport.

ACCEPTANCE EVIDENCE

1. No horizontal document overflow at supported widths.

2. Tap targets meet minimum size and sticky elements do not trap content.

Application states

Credible software spends substantial time loading, empty, stale, partial, degraded, or failed. Each state should explain what remains trustworthy and what the user can do next.

DECISIONS

- Use skeletons only when shape is predictable.
- Preserve last-known-good evidence with a stale marker when appropriate.
- Differentiate no data, filtered-out data, permission denial, and upstream failure.

DELIVERY CONTRACT

1. State matrix for every primary surface.

2. Reusable inline notice, empty state, error boundary, and retry patterns.

ACCEPTANCE EVIDENCE

1. No uncaught promise rejection leaves a blank page.

2. Errors include a safe request ID and recovery action.

Observability model

Observability follows a recommendation request across gateway, feature fetch, retrieval, ranking, re-ranking, and response serialization. Model and data versions are attributes, not inferred later.

DECISIONS

- Adopt OpenTelemetry-compatible trace and metric names.
- Control label cardinality; never use user IDs as metric labels.
- Emit structured events for model fallback and policy degradation.

DELIVERY CONTRACT

1. Span map, metric catalogue, log schema, and local telemetry fixtures.

2. A trace detail view driven by the same schema.

ACCEPTANCE EVIDENCE

1. One seeded request can be reconstructed across all stages.

2. Telemetry tests reject prohibited high-cardinality or sensitive attributes.

Service-level objectives

SLOs translate infrastructure behaviour into product risk. The demo distinguishes target objectives from measured local performance and avoids fake uptime history.

DECISIONS

- Define availability, latency, recommendation completeness, and freshness SLOs.
- Use percentile latency, not averages.
- Attach error-budget policy to risky releases.

DELIVERY CONTRACT

1. SLO catalogue with indicator, target, window, owner, and escalation.

2. Local load-test evidence mapped to the same indicators.

ACCEPTANCE EVIDENCE

1. The UI labels targets, measurements, and simulations distinctly.

2. A failing error budget blocks automatic promotion in the design.

Tracing and logging

Traces explain time and decisions; logs record bounded events. Neither should become an ungoverned copy of feature vectors or behavioural payloads.

DECISIONS

- Sample healthy traces and retain all errors in the production design.
- Redact request context by allowlist.
- Log reason codes and fingerprints instead of raw sensitive values.

DELIVERY CONTRACT

1. Trace waterfall, structured logger, redaction tests, and example queries.

2. Correlation IDs included in API responses and error bodies.

ACCEPTANCE EVIDENCE

1. A seeded slow-feature scenario attributes latency to the correct span.

2. Secrets and synthetic user attributes are absent from captured logs.

Model and data drift

Drift is diagnostic evidence, not proof of model failure. Benchline tracks feature distribution changes, prediction shifts, catalogue mix, and realised quality when labels arrive.

DECISIONS

- Use stable reference windows and minimum sample thresholds.
- Report effect size alongside alert thresholds.
- Separate covariate, prediction, and performance drift.

DELIVERY CONTRACT

1. PSI or Jensen-Shannon checks for selected features, score drift, and slice trends.

2. Alert records linked to feature definitions and affected model bundles.

ACCEPTANCE EVIDENCE

1. Small samples remain ‘insufficient evidence’ rather than green.

2. Synthetic drift fixtures trigger and clear expected alerts.

Threat model

The relevant threats include unauthorised promotion, model artifact substitution, poisoned events, enumeration, debug-data leakage, dependency compromise, and denial of service.

DECISIONS

- Protect release mutations with authorization, idempotency, and audit trails.
- Verify artifact and dataset hashes before loading.
- Bound request size, limit, filter complexity, and expensive debug modes.

DELIVERY CONTRACT

1. STRIDE-style threat register with assets, boundaries, mitigations, and residual risk.

2. Security-focused tests and dependency scanning in CI.

ACCEPTANCE EVIDENCE

1. No high-risk threat lacks an owner or mitigation plan.

2. Tampered model and oversized request fixtures fail safely.

Privacy and responsible personalization

The demo avoids personal data, but the design demonstrates data minimisation, consent awareness, retention, access control, and understandable recommendation reasons.

DECISIONS

- Collect only events required for declared ranking purposes.
- Keep sensitive attributes out of ranking features and logs.
- Support deletion and opt-out as data-pipeline requirements.

DELIVERY CONTRACT

1. Data inventory, retention schedule, consent-state propagation, and deletion workflow design.

2. A model card section describing intended use and affected groups.

ACCEPTANCE EVIDENCE

1. Opted-out actors do not appear in future training snapshots.

2. Explanations use safe reason categories rather than exposing inferred sensitive traits.

Abuse and integrity controls

Recommendation signals can be gamed by bots, creators, or coordinated activity. The platform needs bounded integrity checks even when fraud detection is not its primary purpose.

DECISIONS

- Down-weight suspicious burst activity in offline features.
- Cap single-creator exposure and contribution to trend signals.
- Quarantine schema-valid but behaviourally implausible events for review.

DELIVERY CONTRACT

1. Synthetic abuse scenarios, integrity flags, and a guarded trend feature.

2. Operational notes connecting an alert to affected recommendations.

ACCEPTANCE EVIDENCE

1. A burst attack cannot dominate the trending slate in the seeded test.

2. Integrity interventions remain traceable in candidate provenance.

Infrastructure topology

The local topology minimises prerequisites while preserving production boundaries. Web, API, worker, database, cache, and telemetry can run together; heavyweight substitutes are optional profiles.

DECISIONS

- Use containers for reproducibility but keep core tests runnable outside containers.
- Persist only control-plane data by default.
- Make environment configuration explicit and validated at startup.

DELIVERY CONTRACT

1. Dockerfiles, compose file, health checks, resource limits, and example environment file.

2. Production-oriented deployment manifests kept separate from local defaults.

ACCEPTANCE EVIDENCE

1. Services start in dependency order and become healthy without manual seeding.

2. Missing required configuration fails with a precise message.

Local developer experience

A senior project respects the reviewer’s time. Setup should be discoverable, errors actionable, and common commands memorable.

DECISIONS

- Provide make targets or package scripts for setup, seed, dev, test, benchmark, load-test, and evidence.
- Pin runtime and dependency versions.
- Use pre-commit checks only when they remain fast.

DELIVERY CONTRACT

1. Quickstart, troubleshooting matrix, architecture tour, and sample curl requests.

2. A doctor command that checks versions, ports, files, and optional services.

ACCEPTANCE EVIDENCE

1. A clean checkout reaches the seeded application by following one documented path.

2. No command assumes the author’s absolute filesystem paths.

Continuous integration and delivery

CI protects contracts, numerical correctness, application quality, and supply-chain hygiene. Expensive optional checks are separated from the fast pull-request path.

DECISIONS

- Run formatting, lint, type checks, unit tests, contract tests, UI tests, and build on pull requests.
- Run deterministic benchmark smoke tests with regression tolerances.
- Generate attestable evidence artifacts on tagged releases.

DELIVERY CONTRACT

1. GitHub Actions workflows with caching, least-privilege permissions, and concurrency cancellation.

2. Release checklist and semantic versioning policy.

ACCEPTANCE EVIDENCE

1. The workflow succeeds from the public repository without secrets for core checks.

2. Protected mutations are absent from pull-request workflows.

Testing strategy

Testing follows risk: numerical functions and contracts receive exact unit tests; boundaries receive integration tests; critical workflows receive browser tests; deployment assumptions receive smoke tests.

DECISIONS

- Prefer deterministic fixtures to broad mocks.
- Test fallbacks and failures as seriously as the happy path.
- Keep end-to-end tests few, legible, and business-oriented.

DELIVERY CONTRACT

1. Test matrix by package and risk, shared factories, golden requests, and coverage reporting.

2. Mutation or property-based tests for selected metric and ranking invariants.

ACCEPTANCE EVIDENCE

1. Every release gate has a direct test.

2. Flaky tests are quarantined with an owner and deadline, never silently retried forever.

Data-quality controls

Bad data can improve an offline metric while destroying real behaviour. Controls cover schema, completeness, uniqueness, timeliness, range, distribution, referential integrity, and temporal leakage.

DECISIONS

- Run checks at ingestion, materialisation, and training snapshot boundaries.
- Classify expectations as blocking or diagnostic.
- Store result history and sample only safe failing rows.

DELIVERY CONTRACT

1. Executable expectation suite and a data-quality report in the evidence pack.

2. Fixtures for duplicates, late data, missing catalogue rows, skew, and leakage.

ACCEPTANCE EVIDENCE

1. Blocking failures prevent training and registration.

2. The UI shows affected assets and the last known successful run.

Offline evaluation protocol

Evaluation uses a temporal split that mirrors the next-item decision. Training precedes validation, and validation precedes a locked test window. Candidate generation and ranking are evaluated both separately and end to end.

DECISIONS

- Use impression-aware positives and sampled negatives without crossing time boundaries.
- Tune only on validation; report the test set once per candidate version.
- Fix K values and eligibility rules in a versioned protocol.

DELIVERY CONTRACT

1. Protocol manifest, dataset hashes, seeds, windows, exclusions, and metric configuration.

2. A CLI that rebuilds the complete report from versioned inputs.

ACCEPTANCE EVIDENCE

1. Repeated execution produces matching rows and metrics within strict tolerance.

2. The test report records code and environment versions.

Metric definitions

Metric names are insufficient without semantics. Benchline calculates retrieval, ranking, coverage, diversity, novelty, concentration, calibration where relevant, and operational cost measures.

DECISIONS

- Report Recall@100 for retrieval and NDCG@10 as the primary ranking metric.
- Include MAP@10, MRR@10, hit rate, catalogue coverage, intra-list diversity, novelty, and creator Gini.
- Report inference latency and slate completeness alongside quality.

DELIVERY CONTRACT

1. Pure tested metric functions and a human-readable metric dictionary.

2. Worked examples small enough to verify by hand.

ACCEPTANCE EVIDENCE

1. Metric tests cover empty, duplicate, tied, truncated, and all-relevant cases.

2. Displayed rounding never changes gate evaluation.

Baseline suite

The suite includes random eligible, global popularity, contextual popularity, and collaborative retrieval. Each baseline uses the same eligibility and evaluation protocol as the challenger.

DECISIONS

- Keep random deterministic and clearly weak.
- Do not allow the challenger privileged catalogue or future information.
- Report cost and complexity beside quality.

DELIVERY CONTRACT

1. Baseline implementations, versioned parameters, and comparison table.

2. A failure analysis showing where each baseline remains competitive.

ACCEPTANCE EVIDENCE

1. At least one simple baseline is served in Explorer for honest comparison.

2. Headline improvement always states the named baseline and metric.

Primary backtest

The primary backtest compares the hybrid retrieval plus learned ranker and policy re-ranker against contextual popularity on the locked temporal test window.

DECISIONS

- Pre-register NDCG@10 as primary and coverage, diversity, concentration, and latency as guardrails.
- Evaluate warm, cold-user, cold-item, low-activity, and taxonomy slices.
- Treat the result as portfolio evidence, not a claim about a real business.

DELIVERY CONTRACT

1. CSV and JSON metrics, an HTML/Markdown report, and charts generated from the same data.

2. A concise benchmark card used by the web application.

ACCEPTANCE EVIDENCE

1. The exact command, seed, dataset hash, and runtime are committed.

2. If the challenger loses, the product reports the loss and the plan changes rather than inventing uplift.

Uncertainty and bootstrap intervals

Point estimates hide sample variation. User- or session-level bootstrap intervals quantify uncertainty while preserving within-group dependence.

DECISIONS

- Resample the evaluation unit, not individual recommendation rows.
- Use a fixed seed and at least 1,000 resamples for published evidence.
- Compute intervals for the paired challenger-minus-baseline delta.

DELIVERY CONTRACT

1. Bootstrap implementation, convergence check, and interval table.

2. Visual intervals with zero-delta reference.

ACCEPTANCE EVIDENCE

1. Synthetic identical models produce an interval centred on zero.

2. Claims use ‘improved’ only when the chosen interval excludes zero.

Slice analysis

Average quality can conceal failures for sparse users, new catalogue, price bands, or minority topics. Slices are defined before reading results and include sample sizes.

DECISIONS

- Use behavioural and catalogue slices, not sensitive demographic inference.
- Apply minimum sample rules and multiple-comparison caution.
- Investigate large regressions even when not statistically conclusive.

DELIVERY CONTRACT

1. Slice definition registry, metric table, and worst-slice attention queue.

2. Explorer presets for representative failure cases.

ACCEPTANCE EVIDENCE

1. Every slice result includes N and evaluation window.

2. Small slices are labelled exploratory rather than silently dropped.

Ablation plan

Ablations prove which system components earn their complexity. Candidate blending, affinity features, quality features, recency, and policy re-ranking are removed one at a time.

DECISIONS

- Run ablations from the same trained/evaluation protocol where methodologically valid.
- Report both relevance and guardrail changes.
- Include serving cost when a feature group affects latency.

DELIVERY CONTRACT

1. Ablation configuration and ranked contribution table.

2. Narrative explaining retained and rejected components.

ACCEPTANCE EVIDENCE

1. A component with no measurable value is removed or explicitly justified.

2. The UI does not confuse feature importance with causal impact.

Simulated online experiment

A transparent replay simulation demonstrates experiment mechanics without pretending to be real user traffic. It covers assignment, exposure, outcome generation, guardrails, and sequential peeking risk.

DECISIONS

- Hash users into stable control and treatment groups.
- State all response-model assumptions.
- Label outputs ‘simulation’ at every presentation layer.

DELIVERY CONTRACT

1. Simulation notebook or script, sample-ratio-mismatch check, effect estimate, interval, and guardrails.

2. An experiment-readout view using the resulting artifact.

ACCEPTANCE EVIDENCE

1. Assignment is stable and approximately balanced.

2. Changing the assumed treatment effect changes outcomes in the expected direction.

Performance and load testing

The serving claim must be measured under a declared machine, concurrency, request mix, and warm-up. The goal is honest engineering evidence, not a borrowed hyperscale number.

DECISIONS

- Measure p50, p95, p99, throughput, error rate, and fallback rate.
- Use a warm and cold-cache scenario.
- Keep test duration sufficient for stable percentiles but practical for local reruns.

DELIVERY CONTRACT

1. Load generator, environment manifest, raw samples, summary JSON, and chart.

2. Budget breakdown for feature, retrieval, rank, policy, and serialization stages.

ACCEPTANCE EVIDENCE

1. The README reports hardware and exact command beside latency figures.

2. Performance regression thresholds run in a controlled optional CI job.

Failure and recovery design

The recommendation path should degrade usefully when online features, an index, a ranker, or telemetry becomes unavailable. Recovery behaviour is part of product quality.

DECISIONS

- Define a fallback ladder: champion, previous champion, contextual popularity, global eligible popularity.
- Set timeouts and circuit-breaking at dependency boundaries.
- Return reason codes and degraded status without exposing internals.

DELIVERY CONTRACT

1. Failure matrix, injected-fault fixtures, recovery metrics, and operator runbooks.

2. A visible degraded-state example in Explorer and Delivery.

ACCEPTANCE EVIDENCE

1. Every single dependency failure has a bounded response path.

2. Recovery tests assert slate validity and audit emission.

Promotion and rollback

A release is a state transition backed by evidence. Promotion requires compatible contracts, passing gates, an expected champion version, and a human note; rollback is always available.

DECISIONS

- Use champion/challenger aliases and immutable bundles.
- Warm and health-check a candidate before alias change.
- Retain release evidence and the previous alias target.

DELIVERY CONTRACT

1. Gate engine, signed release record, atomic alias update, rollback endpoint, and drill.

2. UI confirmation that summarises consequence without theatrical warnings.

ACCEPTANCE EVIDENCE

1. Concurrent promotions cannot overwrite one another silently.

2. The rollback drill records a measured recovery time and restored version.

Phase 1

credible vertical slice

Phase 1 establishes the full contract with simple algorithms: seeded events, point-in-time features, popularity and collaborative candidates, deterministic scoring, API serving, Explorer, and baseline evidence.

DECISIONS

- Prioritise correctness and explainability over model sophistication.
- Build shared contracts before parallel UI and analytics work.
- Ship one polished path rather than six disconnected screens.

DELIVERY CONTRACT

1. Repository foundation, generator, feature pipeline, baseline models, recommendation endpoint, Explorer, tests, and CI.

2. A first evidence pack with real computed metrics.

ACCEPTANCE EVIDENCE

1. A reviewer can complete the signature demo from a clean clone.

2. All claims on the landing page link to generated evidence.

Phase 2

evaluation and operations

Phase 2 adds learned ranking, policy re-ranking, experiment comparisons, registry workflows, feature health, delivery evidence, and browser-grade application states.

DECISIONS

- Add complexity only behind stable interfaces from Phase 1.
- Promote the challenger only after quality and guardrail gates pass.
- Use incident fixtures to prove diagnosis workflows.

DELIVERY CONTRACT

1. Experiments, Registry, Feature Health, Delivery, release gates, telemetry, and expanded tests.

2. Ablations, bootstrap intervals, slices, and load-test evidence.

ACCEPTANCE EVIDENCE

1. The traceability matrix is at least 90 percent complete for P0 requirements.

2. All primary routes pass responsive and accessibility QA.

Phase 3

production substitutions

Phase 3 documents and optionally demonstrates replacements for local adapters: streaming ingestion, distributed materialisation, managed feature storage, ANN search, model registry, telemetry backend, and orchestration.

DECISIONS

- Preserve domain contracts while replacing infrastructure adapters.
- Do not make optional services prerequisites for interview review.
- Benchmark each substitution against the local reference.

DELIVERY CONTRACT

1. Adapter implementations or design spikes, compose profiles, Kubernetes manifests, and migration notes.

2. Cost, reliability, and operational trade-off records.

ACCEPTANCE EVIDENCE

1. The core test suite runs against at least one alternate adapter where feasible.

2. Production diagrams match actual ports in code.

Plan-to-code traceability

The final product must prove alignment with this document. Traceability links each P0 requirement to implementation, tests, evidence, and status, making omissions visible.

DECISIONS

- Use stable requirement IDs derived from page and item number.
- Allow one artifact to satisfy multiple requirements but never leave proof implicit.
- Mark deferred items with rationale and roadmap destination.

DELIVERY CONTRACT

1. A machine-readable traceability file and rendered Markdown table.

2. A validation script that fails on missing P0 evidence.

ACCEPTANCE EVIDENCE

1. All P0 rows are implemented or explicitly accepted as deviations before release.

2. Links resolve in the public repository.

Source references and final mandate

The design is grounded in current role expectations and primary technical guidance. Spotify's current personalization roles emphasise end-to-end ML systems, evaluation, experimentation, distributed data, and operational excellence. Google documents multi-stage recommendation architectures; Feast documents point-in-time joins; MLflow documents alias-based registry workflows; Evidently documents ranking metrics.

DECISIONS

- Primary references: lifeatspotify.com/jobs; developers.google.com/machine-learning/recommendation; docs.cloud.google.com/architecture/implement-two-tower-retrieval-large-scale-candidate-generation; docs.feast.dev/master/getting-started/concepts/point-in-time-joins; mlflow.org/docs/latest/ml/model-registry/workflow; docs.evidentlyai.com/metrics/all_metrics.
- Treat external patterns as guidance, not copied interface or code.
- Update dated assumptions when the implementation materially changes.

DELIVERY CONTRACT

1. Build the smallest complete system that proves every critical claim, then deepen it without breaking reproducibility.

2. Publish only after the plan-to-code audit, numerical validation, browser QA, and clean-clone check pass.

ACCEPTANCE EVIDENCE

1. The final repository is honest about synthetic data and local measurements.

2. The product earns credibility through connected evidence, not visual polish alone.